

Advanced Resources

Data sources, explicit dependencies,
dynamic blocks and the resource
lifecycle — the tools that make real-world
code robust.

Data sources

depends_on

Dynamic blocks

Lifecycle

Lab 07

Today's roadmap



1 • Data sources

Query existing resources instead of recreating them.



2 • Explicit dependencies

When the graph can't infer order — `depends_on`.



3 • Dynamic blocks

Generate repeatable nested blocks from data.



4 • Lifecycle & Lab 07

Control create/destroy; finish the app tier.

01

Data Sources

Read the world instead of rebuilding it.

Data sources

Fetch information about existing resources or the provider itself — resolved at `plan` time.

- Reuse infra managed elsewhere
- Fetch dynamic values (IDs, latest image...)
- Reference like resources:
`data.type.name.attr`

HCL

```
data "azurerm_client_config" "current" {}

resource "azurerm_key_vault" "kv" {
  tenant_id = data.azurerm_client_config.current.tenant_id
  # ...
}
```

👁 They only **read** — never create or manage.

Using data sources well



Filter precisely

Query by tag, name or id — don't fetch more than you need.



Use sparingly

Every data source is a lookup at plan time — too many slow you down.



Document usage

Make it clear why a lookup exists for the next maintainer.

02

Dependencies, Dynamic Blocks & Lifecycle

Fine control over how Terraform builds and changes things.

Explicit dependencies

Terraform infers order from references. Sometimes a hidden dependency needs to be stated.

- Use when B relies on A's **behavior** but not its data
- `depends_on` forces A to finish first
- Last resort — it makes plans more conservative

HCL

```
resource "azurerm_role_assignment" "kv" {  
  # ...  
  depends_on = [  
    azurerm_key_vault.kv  
  ]  
}
```

Dynamic blocks

Some resources take a **nested block repeated many times** — NSG rules, App Service IP restrictions, tags. Writing them by hand is verbose and drifts. A **dynamic** block generates them from a collection.

HCL · BY HAND

```
resource "azurerm_network_security_group" "nsg" {  
  security_rule { name = "http" # ... }  
  security_rule { name = "https" # ... }  
  security_rule { name = "ssh" # ... }  
  # one more port = one more copy-pasted block  
}
```

- **DRY** — one block instead of N copy-pasted ones
- **Data-driven** — add or remove a rule by editing a list, not the resource
- **Consistent** — every generated block has the same shape, so no drift

⚠ Only reach for it when a block **truly** repeats — a single static block is easier to read.

Dynamic blocks — syntax

- **label** — the nested block to generate (`security_rule`)
- **for_each** — the collection to loop over
- **content** — the body built for each item
- Reference items via the label: `security_rule.key` / `.value` (rename with `iterator`)

HCL

```
locals {
  ports = [80, 443, 8080]
}

resource "azurerm_network_security_group" "nsg" {
  name = "app-nsg"

  dynamic "security_rule" {
    for_each = local.ports
    content {
      name                = "allow-${security_rule.value}"
      priority            = 100 + security_rule.key
      destination_port_range = security_rule.value
      access              = "Allow"
    }
  }
}
```

The lifecycle block

- **create_before_destroy** — build the replacement first, then remove the old one (no downtime)
- **prevent_destroy** — reject any plan that would destroy this resource
- **ignore_changes** — let an external process own certain attributes (e.g. tags)



`ignore_changes = [tags]` is exactly why your labs say "ignore manual tag changes."

HCL

```
resource "azurerm_resource_group" "app" {  
  name      = "app-rg"  
  location = "westeurope"  
  
  lifecycle {  
    prevent_destroy = true    # block deletion  
    ignore_changes  = [tags]  # portal owns tags  
  }  
}
```

 LIVE DEMO

Data sources & lifecycle in action

Add a data source and a `lifecycle` block to the Azure project, then plan to see how behavior changes.

Provider · azurerm

■ LAB 07 · 30-40 MIN

App Services & PostgreSQL

Finish the application tier — App Services on a plan and a PostgreSQL database — using data sources and `ignore_changes` on tags.

You'll practise: data sources · dependencies · `lifecycle` · dynamic blocks

Key takeaways

✓ Data sources read

Reuse existing infra & dynamic values.

✓ depends_on is a last resort

For hidden, behavior-only dependencies.

✓ Lifecycle gives control

Zero-downtime, protection, drift tolerance.

✓ Next up

Session 08 — importing existing infra.



Questions?

Up next — [Session 08: Importing Existing Infrastructure.](#)